

BÁO CÁO PHÂN TÍCH VÀ KHẮC PHỤC LỖI RESTTEMPLATE

1. Phân Tích 3 Lỗi Trong Đoạn Code Gốc và Hậu Quả

Lỗi 1: Tự khởi tạo RestTemplate bằng toán tử `new` thay vì dùng `@LoadBalanced` Bean

- **Mô tả:** Trong code gốc, developer sử dụng `private final RestTemplate restTemplate = new RestTemplate();` trực tiếp trong component.
- **Hậu quả:** Đối tượng `RestTemplate` tự khởi tạo này không có khả năng phân giải tên dịch vụ (service ID) thông qua Eureka Service Registry. Khi có nhiều instance của `product-service` hoạt động và tự động scale, `RestTemplate` này không thể load balance (phân phối tải) vòng tròn giữa các instance, dẫn tới mất tính chất high-availability (HA).

Lỗi 2: Hardcode địa chỉ IP và Port dịch vụ (`http://192.168.1.45:8082`)

- **Mô tả:** Endpoint gọi tới microservice được khai báo dạng chuỗi cố định chứa IP tĩnh.
- **Hậu quả:** Trong môi trường Kubernetes, Cloud hoặc Staging, các Container/Pod thường xuyên thay đổi địa chỉ IP khi khởi động lại hoặc khi mở rộng/thu hẹp tài nguyên (auto-scaling). Việc hardcode IP dẫn đến lỗi kết nối ngay lập tức (`ConnectException`) khi môi trường deploy thay đổi, làm tê liệt tính năng hiển thị thông tin sản phẩm.

Lỗi 3: Không cấu hình Timeout cho RestTemplate

- **Mô tả:** Không thiết lập `connectTimeout` và `readTimeout` cho kết nối TCP/HTTP.
- **Hậu quả:** Mặc định, RestTemplate sử dụng kết nối vô hạn hoặc thời gian timeout rất lớn của hệ điều hành. Khi một instance của `product-service` bị treo hoặc quá tải (chạy quá chậm), các thread của `order-service` sẽ bị block để chờ phản hồi. Nếu số lượng request đồng thời tăng cao, toàn bộ luồng xử lý (thread pool) của `order-service` sẽ bị nghẽn (thread starvation), dẫn tới sụp đổ toàn bộ dịch vụ (Cascading Failure).

2. Giải Pháp Khắc Phục

1. Cấu hình LoadBalanced RestTemplate Bean:

- Tạo một class cấu hình `RestTemplateConfig` chứa một `@Bean` của `RestTemplate`.
- Sử dụng decorator `@LoadBalanced` để kích hoạt bộ lọc của Spring Cloud LoadBalancer, cho phép giải quyết URL chứa `service-id` từ Eureka Server.

2. Thiết lập Timeout hợp lý:

- Sử dụng `RestTemplateBuilder` để thiết lập `connectTimeout` là 2 giây và `readTimeout` là 3 giây để đảm bảo hệ thống giải phóng luồng nhanh chóng nếu service đích gặp sự cố.

3. Thay thế URL bằng Service-ID:

- Chuyển URL đích thành `http://product-service/api/products/{id}`. Spring Cloud sẽ tự động map `product-service` thành IP thực tế của instance đang hoạt động khỏe mạnh.

4. Xử lý ngoại lệ và Fallback:

- Bắt `HttpClientErrorException.NotFound` (404) để ném ra một ngoại lệ nghiệp vụ cụ thể `ProductNotFoundException`.
- Bắt `ResourceAccessException` (timeout/mạng lỗi) để kích hoạt fallback, trả về thông tin sản phẩm tạm thời để nâng cao độ tin cậy của hệ thống (Graceful Degradation).

3. Kết Quả Kiểm Thử (Unit Test)

- Toàn bộ 3 kịch bản kiểm thử (Gọi thành công, Lỗi 404 sinh ra Exception, Lỗi Timeout kích hoạt fallback) đã được kiểm thử thành công bằng thư viện JUnit và Mockito, đảm bảo tính ổn định tối đa trước khi đưa lên môi trường Production.